

PROJECT REPORT: CUSTOMER RELATIONSHIP MANAGEMENT USING DJANGO

By

SAMANWAYA

BHATTACHARYA

ID:MST01-0071

Trainer – Farzana Rashid



Meta Scifor Technologies Pvt. Ltd.

No.13-14, 2nd floor, Kothnur Main Rd, JP Nagar 7th Phase, Bengaluru Karnataka 560078

Abstract

The Django Customer Relationship Management (CRM) project is a comprehensive solution designed to streamline and enhance business interactions with customers. Built on the Django framework, it offers robust features including efficient customer data management, seamless communication tools, and insightful analytics. The platform prioritizes scalability, security, and user-friendliness, enabling businesses to foster stronger relationships, improve customer retention, and boost overall efficiency.

In summary, the Django CRM project combines advanced technology, thoughtful development, and customer-focused design, empowering businesses to optimize their customer management strategies and drive growth in a competitive marketplace.

Table of Contents:

- 1.Introduction
- 2.Project Overview
- 3.Technologies Used
- 4.System Architecture
- 5.Implementation Details
- 6.Challenges Faced
- 7.Future Enhancements
- 8.Conclusion
- 9.References

1. Introduction:

The **Customer Relationship Management (CRM)** project is a business solution developed using the Django framework. It incorporates key sections such as Login Page, Signup Page, Dashboard, Client Management, Lead Tracking, Team Management to offer businesses an all-encompassing CRM experience.

With Django's built-in features like ORM (Object-Relational Mapping), authentication, and security measures, along with its rich ecosystem of third-party libraries and plugins, developing a dynamic and secure CRM system becomes efficient and scalable.

From managing customer data and tracking sales leads to automating communication and analyzing performance, Django empowers developers to build intuitive, high-performance CRM platforms that help businesses nurture relationships, improve retention, and drive growth.

2. Project Overview:

The project integrates Django framework along with TailwindCSS for frontend styling and responsiveness. It features an enhanced Login authorization interface using Django inbuilt authorization for easy customization. Key functionalities include Login/Signup authorization, Clients Management, Keep track of Leads, Adding comments, Exporting all client details, Manage Teams and User account details which are provided using Django's built-in internationalization packages and middleware.

3. Technologies Used:

- **Django 5.0.1:** Django is a high-level Python web framework that facilitates rapid development of secure and maintainable websites and web applications. It follows the model-view-template (MVT) architectural pattern.
- **TailwindCSS CDN:** It is a way to include TailwindCSS directly into your project without the need for installation or compilation. By adding the CDN link to your HTML file, you can immediately start using Tailwind's utility-first CSS classes to style your elements.
- **SQLite3:** It is the default database engine in Django, a lightweight, file-based relational database. It requires no separate server installation and is ideal for development or small projects, as it stores data in a single file and is easy to set up.
- **Django Middleware:** It is a framework component that processes requests and responses. It sits between the client and the view, allowing you to modify requests before they reach the view and alter responses before they're sent back to the client.
- **HTML:** It stands for Hyper Text Markup Language. It is the standard markup language for creating Web pages. It describes the structure of a Web page. It consists of a series of elements. Its elements tell the browser how to display the content.
- **CSS:** CSS (Cascading Style Sheets) is a stylesheet language used to style the presentation and layout of HTML documents. It defines the styles, such as colors, fonts, margins, and padding, to enhance the visual appearance of web pages.

4. System Architecture:

The system architecture of the **Customer Relationship Management (CRM) system** follows Django's MVT (Model-View-Template) pattern, ensuring a clean separation of concerns. Models are used for managing customer data, sales records, and interactions within a relational database. Views handle user requests, process business logic, and interact with models to fetch or update data, while templates are responsible for rendering dynamic HTML content. Key middleware such as `django.middleware.security.SecurityMiddleware` is employed to enhance security, and `django.middleware.locale.LocaleMiddleware` enables localization for multi-lingual support, ensuring a secure and globally adaptable CRM solution.

5. Implementation Details:

Index Page: Displays the homepage of a Customer Relationship Management (CRM) Solution web application. It has a dark-themed header with the project title, a navigation bar including links to [About](#), [Sign Up](#), and [Log In](#), and a prominent [Get Started](#) button. The tagline encourages users to efficiently manage leads, convert them into clients, and drive business growth. Below this section, there's an introduction titled "[What is CRM?](#)", providing a brief explanation of customer relationship management, its practices, and technologies using HTML and TailwindCSS and Django.

Signup Page: The webpage shows a simple "Sign Up" form with the following fields:

- Username
- Email
- Password (Password1)
- Confirm Password (Password2)

Below the form, there are two buttons: "Sign Up" and "Log In," the latter is for users already registered.

Login Page: The login page serves as the entry point for users to access their accounts. It typically includes fields for users to input their username/email and password.

Dashboard: The webpage shows a "Dashboard" with two sections:

- [Current Leads](#) on the left, listing 5 current leads of the user and team each with a "Details >>" link.
- [Newest Clients](#) on the right, listing listing 5 new clients of the user and team each with a "Details >>" link.
- The navigation bar at the top includes links to [About](#), [Team Name](#), [Leads](#), [Clients](#), [Dashboard](#), and [My account](#).

Lead List: Displays the list of all leads and a [Add Lead](#) button at the top to add a lead.

Client List: Displays the list of all clients and [Add Client](#) button to add a client and [Export Client](#) button to exports the details of all clients into csv file.

Team Details: Displays the active team details (like owner name, members) and a [Switch Team](#) button for switching between teams. Also include a [Edit](#) button if the user is the owner of the team.

My Account: Displays the name of the user and the active team name and Logout button below to log out of the user account. Also include a Edit button beside team name, if the user is the owner of the team.

Index Page:

Customer Relationship Management

AboutTeam: SpartanLeadsClientsDashboardMy account

Welcome to Your CRM Solution

Efficiently manage your Leads. Convert them into Clients. Drive your business forward with ease.

Get Started

What is CRM?

CRM (customer relationship management) is the combination of practices, strategies and technologies that companies use to manage and analyze customer interactions and data throughout the customer lifecycle.

Customer relationship management (CRM) includes the principles, practices, and guidelines that an organization follows when interacting with its customers.

CRM is often used to refer to technology companies and systems that help manage external interactions with customers.Major areas of growth in CRM technology include software, cloud computing, and artificial intelligence.

© 2024 CRM. All rights reserved.

Signup Page:

Customer Relationship Management

AboutSign UpLog in

Sign Up

Username:

Your username

Email:

Your email address

Password1:

Your password

Password2:

Confirm password

Sign Up

Already registered?

Log In

© 2024 CRM. All rights reserved.

Login Page:

Customer Relationship Management

AboutSign UpLog In

Log In

Username:

Your username

Password:

Your password

Log in

Don't have an account? Register here.

Sign Up

© 2024 CRM. All rights reserved.

Dashboard:

Customer Relationship Management

AboutTeam: SpartanLeadsClientsDashboardMy account

Dashboard

Current Leads

test 3
Details >>

Test II
Details >>

TestI
Details >>

Newest Clients

Test client
Details >>

Test lead 4
Details >>

Test Lead III edited
Details >>

New Lead
Details >>

Test Lead edited client
Details >>

© 2024 CRM. All rights reserved.

Lead Lists:

Customer Relationship Management

About

Team: Spartan

Leads

Clients

Dashboard

My account

Leads

Add Lead

Name	Priority	Status
Test II	Medium	New
Test1	High	Contacted
test 3	Low	Lost

© 2024 CRM. All rights reserved.

Clients Lists:

Customer Relationship Management

About

Team: Spartan

Leads

Clients

Dashboard

My account

Clients

Add Client

Export Clients

Name
New Lead
Test Lead III edited
Test Lead edited client
Test client
Test lead 4

© 2024 CRM. All rights reserved.

Team Details:

Customer Relationship Management

AboutTeam: SpartanLeadsClientsDashboardMy account

Spartan

Switch team

Edit

Team owner: admin

Team members:
admin
samanwaya

© 2024 CRM. All rights reserved.

My Account:

Customer Relationship Management

AboutTeam: SpartanLeadsClientsDashboardMy account

My Account

Username: admin

Team: Spartan

Edit

Log out

© 2024 CRM. All rights reserved.

Add Lead:

Customer Relationship Management

About

Team: Spartan

Leads

Clients

Dashboard

My account

Add lead

Name:

Email:

Description:

Priority:

Medium

Status:

New

Save

© 2024 CRM. All rights reserved.

Add Client:

Customer Relationship Management

About

Team: Spartan

Leads

Clients

Dashboard

My account

Add Client

Present Clients count: 5

Name:

Email:

Description:

Add

© 2024 CRM. All rights reserved.

Lead Details:

Customer Relationship Management

About

Team: Spartan

Leads

Clients

Dashboard

My account

test 3

Description
test lead

Status
Lost

Priority
Low

Creation Date & Time
Oct-21-2024 10:32

Modification Date & Time
Oct-21-2024 10:32

Edit

Delete

Convert to a client

Comments

Content:

Submit

<< Back to Leads

© 2024 CRM. All rights reserved.

Client Details:

Customer Relationship Management

About

Team: Spartan

Leads

Clients

Dashboard

My account

New Lead

Email
CAFscf@djad.csdc

Creation Date & Time
Oct-20-2024 03:57

Modification Date & Time
Oct-20-2024 03:57

Comments

Content:

Submit

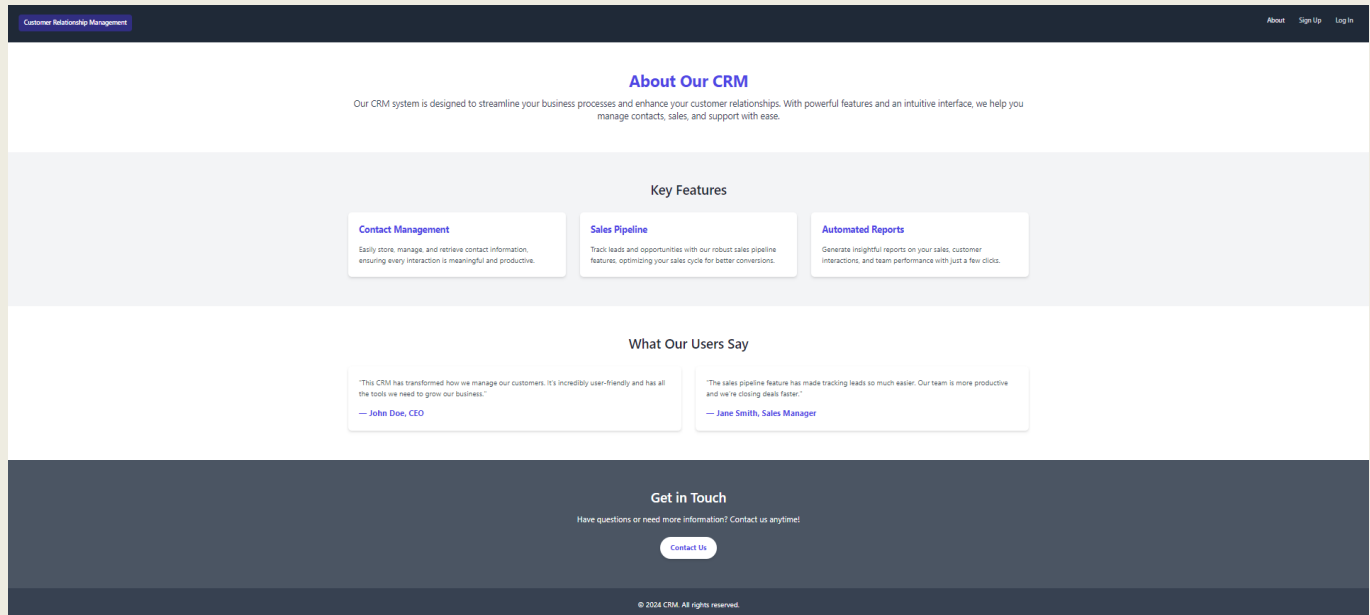
Edit

Delete

<< Back to Clients

© 2024 CRM. All rights reserved.

About:



Source Code:

```
from django.shortcuts import render
from django.contrib.auth import logout
from django.shortcuts import redirect

def logout_view(request):
    logout(request)
    return redirect('index')

# Create your views here.
def index(request):
    return render(request, "core/index.html")
def about(request):
    return render(request, "core/about.html")
```

```
from django.contrib import admin
from django.urls import path, include
from core.views import index, about, logout_view
from django.contrib.auth import views
from userprofile.forms import LoginForm

urlpatterns = [
    path('admin/', admin.site.urls),
    path('dashboard/', include('dashboard.urls')),
    path('dashboard/leads/', include('lead.urls')),
    path('dashboard/clients/', include('client.urls')),
    path('dashboard/', include('userprofile.urls')),
```

```

    path('', index, name='index'),
    path('about/', about, name='about'),
    path('log-out/', logout_view, name='logout'),
    path('log-in/', views.LoginView.as_view(template_name='userprofile/login.html',
authentication_form=LoginForm), name='login'),

    path('log-in/', views.LoginView.as_view(template_name='userprofile/login.html'),
name='login'),
]

```

```

from django.shortcuts import render, redirect
from .forms import SignupForm
from .models import Userprofile
from django.contrib.auth.decorators import login_required
from team.models import Team
# Create your views here.

def signup(request):
    if request.method == 'POST':
        form = SignupForm(request.POST)

        if form.is_valid():
            user = form.save()

            team = Team.objects.create(name='The team name', created_by=user)
            team.members.add(user)
            team.save()

            Userprofile.objects.create(user=user, active_team=team)

            return redirect('/log-in/')
    else:
        form = SignupForm()

    return render(request, 'userprofile/signup.html', {
        'form': form
    })

@login_required
def myaccount(request):
    return render(request, 'userprofile/myaccount.html')

```

```

from django.shortcuts import render
from django.contrib.auth.decorators import login_required
from lead.models import Lead
from client.models import Client
# Create your views here.
@login_required
def dashboard(request):
    team = request.user.userprofile.active_team
    leads = Lead.objects.filter(team=team, converted_to_client=False).order_by('-created_at')[0:5]
    clients = Client.objects.filter(team=team).order_by('-created_at')[0:5]

    return render(request, 'dashboard/dashboard.html', {
        'leads': leads,
        'clients': clients
    })

```

```

from django.urls import path
from . import views

app_name = 'dashboard'

urlpatterns = [
    path('', views.dashboard, name='index'),
]

```

```

from django.urls import path
from . import views

app_name = 'leads'

urlpatterns = [
    path('add/', views.LeadCreateView.as_view(), name='add'),
    path('', views.LeadListView.as_view(), name='list'),
    path('<int:pk>/', views.LeadDetailView.as_view(), name='detail'),
    path('<int:pk>/delete/', views.LeadDeleteView.as_view(), name='delete'),
    path('<int:pk>/convert/', views.ConvertToClientView.as_view(), name='convert'),
    path('<int:pk>/add-comment/', views.AddCommentView.as_view(), name='add_comment'),
    path('<int:pk>/edit/', views.LeadUpdateView.as_view(), name='edit'),
]

```

```

from django.shortcuts import redirect, get_object_or_404
from django.contrib.auth.mixins import LoginRequiredMixin
from .models import Lead
from .forms import AddCommentForm
from django.urls import reverse_lazy
from client.models import Client, Comment as ClientComment
from django.views.generic import
ListView, DetailView, DeleteView, UpdateView, CreateView, View
from django.contrib.messages.views import SuccessMessageMixin
from django.contrib.messages import add_message, INFO
# Create your views here.

class LeadListView(LoginRequiredMixin, ListView):

    model = Lead

    def get_queryset(self):
        queryset = super(LeadListView, self).get_queryset()
        active_team = self.request.user.userprofile.active_team
        return
        queryset.filter(created_by=self.request.user, converted_to_client=False, team=active_team)

class LeadDetailView(LoginRequiredMixin, DetailView):
    model = Lead

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['form'] = AddCommentForm()
        return context

    def get_queryset(self):
        queryset = super(LeadDetailView, self).get_queryset()
        team = self.request.user.userprofile.active_team

        return queryset.filter(team=team, pk=self.kwargs.get('pk'))

```

```

model = Lead
success_url = reverse_lazy('leads:list')
success_message = 'Lead Deleted Successfully!'

def get_queryset(self):
    queryset = super(LeadDeleteView, self).get_queryset()
    team = self.request.user.userprofile.active_team

    return queryset.filter(team=team, pk=self.kwargs.get('pk'))

def get(self, request, *args, **kwargs):
    return self.post(request, *args, **kwargs)
class LeadCreateView(LoginRequiredMixin, CreateView):
    model = Lead
    fields = ('name', 'email', 'description', 'priority', 'status',)
    success_url = reverse_lazy('leads:list')

    def form_valid(self, form):
        self.object = form.save(commit=False)
        self.object.created_by = self.request.user
        self.object.team = self.request.user.userprofile.get_active_team()
        self.object.save()
        add_message(self.request, INFO, f"Lead '{self.object.name}' created successfully!")
        return redirect(self.get_success_url())

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        team = self.request.user.userprofile.get_active_team()
        context['team'] = team
        context['title'] = 'Add lead'

        return context
class LeadUpdateView(LoginRequiredMixin, SuccessMessageMixin, UpdateView):
    model = Lead
    fields = ('name', 'email', 'description', 'priority', 'status',)
    success_url = reverse_lazy('leads:list')
    success_message = 'Lead Edited Successfully'

    def get_queryset(self):
        queryset = super(LeadUpdateView, self).get_queryset()

        return queryset.filter(created_by=self.request.user, pk=self.kwargs.get('pk'))

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['title'] = 'Edit lead'

        return context
class ConvertToClientView(LoginRequiredMixin, View):
    def get(self, request, *args, **kwargs):
        pk = self.kwargs.get('pk')

        team = self.request.user.userprofile.active_team
        lead = get_object_or_404(Lead, team=team, pk=pk)
        team = self.request.user.userprofile.get_active_team()

        client = Client.objects.create(
            name=lead.name,
            email=lead.email,

```

```

        created_by=request.user,
        team=team,
    )

    lead.converted_to_client = True
    lead.save()
    comments = lead.comments.all()
    for comment in comments:
        ClientComment.objects.create(
            client = client,
            content = comment.content,
            team = team,
            created_by = comment.created_by,
        )
    add_message(self.request, INFO, f"Lead '{lead.name}' converted to client successfully!")
    return redirect('leads:list')
class AddCommentView(LoginRequiredMixin,View):

    def post(self, request, *args, **kwargs):
        pk = kwargs.get('pk')

        form = AddCommentForm(request.POST)

        if form.is_valid():
            comment = form.save(commit=False)
            comment.team = self.request.user.userprofile.get_active_team()
            comment.created_by = request.user
            comment.lead_id = pk
            comment.save()

        return redirect('leads:detail', pk=pk)

```

```

from django.db import models
from django.contrib.auth.models import User
from team.models import Team

# Create your models here.
class Lead(models.Model):
    LOW = 'low'
    MEDIUM = 'medium'
    HIGH = 'high'
    CHOICES_PRIORITY = (
        (LOW, 'Low'),
        (MEDIUM, 'Medium'),
        (HIGH, 'High')
    )
    NEW = 'new'
    CONTACTED = 'contacted'
    WON = 'won'
    LOST = 'lost'
    CHOICES_STATUS = (
        (NEW, 'New'),
        (CONTACTED, 'Contacted'),
        (WON, 'Won'),
        (LOST, 'Lost')
    )
    team = models.ForeignKey(Team, related_name='leads', on_delete=models.CASCADE)
    name = models.CharField(max_length=255)
    email = models.EmailField()
    description = models.TextField(blank=True, null=True)

```

```

priority = models.CharField(max_length=10,choices=CHOICES_PRIORITY,default=MEDIUM)
status = models.CharField(max_length=10,choices=CHOICES_STATUS,default=NEW)
converted_to_client = models.BooleanField(default=False)
created_by = models.ForeignKey(User,related_name='leads', on_delete=models.CASCADE)
created_at = models.DateTimeField(auto_now_add=True)
modified_at = models.DateTimeField(auto_now=True)

class Meta:
    ordering = ('name',)

def __str__(self):
    return self.name

class Comment(models.Model):
    team = models.ForeignKey(Team, related_name='lead_comments',
on_delete=models.CASCADE)
    lead = models.ForeignKey(Lead,related_name='comments',on_delete=models.CASCADE)
    content = models.TextField(blank=True, null=True)
    created_by = models.ForeignKey(User, related_name='lead_comments',
on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.created_by.username

```

```

from django import forms
from .models import Lead,Comment

class AddLeadForm(forms.ModelForm):
    class Meta:
        model = Lead
        fields = ('name', 'email', 'description', 'priority', 'status')
class AddCommentForm(forms.ModelForm):
    class Meta:
        model = Comment
        fields = ('content',)

```

```

from django.urls import path
from . import views

app_name = 'clients'
urlpatterns = [
    path('',views.clients_list,name='list'),
    path('<int:pk>/',views.clients_detail,name='detail'),
    path('add/',views.clients_add,name='add'),
    path('<int:pk>/delete_client/',views.clients_delete,name='delete'),
    path('<int:pk>/edit_client/',views.clients_edit,name='edit'),
    path('<int:pk>/add-comment/',views.clients_detail,name='add_comment'),
    path('export/',views.client_export,name='export'),
]

```

```

from django.db import models
from django.contrib.auth.models import User
from team.models import Team
# Create your models here.
class Client(models.Model):
    team = models.ForeignKey(Team, related_name='clients', on_delete=models.CASCADE)
    name = models.CharField(max_length=255)
    email = models.EmailField()
    description = models.TextField(blank=True, null=True)
    created_by = models.ForeignKey(User, related_name='clients',

```

```

on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)
    modified_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ('name',)

    def __str__(self):
        return self.name

class Comment(models.Model):
    team = models.ForeignKey(Team, related_name='client_comments',
on_delete=models.CASCADE)
    client = models.ForeignKey(Client, related_name='comments', on_delete=models.CASCADE)
    content = models.TextField(blank=True, null=True)
    created_by = models.ForeignKey(User, related_name='client_comments',
on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.created_by.username

from django import forms
from .models import Client, Comment

class AddClientForm(forms.ModelForm):
    class Meta:
        model = Client
        fields = ('name', 'email', 'description')

class AddCommentForm(forms.ModelForm):
    class Meta:
        model = Comment
        fields = ('content',)

```

```

from django.shortcuts import render, get_object_or_404, redirect
from django.contrib.auth.decorators import login_required
from .models import Client
from django.contrib import messages
from .forms import AddClientForm, AddCommentForm
import csv
from django.http import HttpResponse
# Create your views here.
@login_required
def client_export(request):
    clients = Client.objects.filter(created_by=request.user)
    response = HttpResponse(content_type="text/csv",
        headers={"Content-Disposition": 'attachment; filename="clients.csv"'},)
    writer = csv.writer(response)
    writer.writerow(["Name", "Description", "Created at", "Created by"])
    for client in clients:
        writer.writerow([client.name, client.description, client.created_at,
client.created_by])
    return response
@login_required
def clients_list(request):
    active_team = request.user.userprofile.active_team
    clients = Client.objects.filter(created_by=request.user, team=active_team)
    return render(request, 'client/clients_list.html', {
        'clients': clients

```

```

    })
@login_required
def clients_detail(request, pk):
    client = get_object_or_404(Client, created_by = request.user, pk=pk)
    if request.method == 'POST':
        form = AddCommentForm(request.POST)
        if form.is_valid():
            comment = form.save(commit=False)
            comment.team = request.user.userprofile.active_team
            comment.created_by = request.user
            comment.client = client
            comment.save()
            return redirect('clients:detail', pk=pk)
    else:
        form = AddCommentForm()

    return render(request, 'client/clients_detail.html', {
        'client': client,
        'form': form
    })

@login_required
def clients_add(request):
    team = request.user.userprofile.active_team
    if request.method == 'POST':
        form = AddClientForm(request.POST)
        if form.is_valid():

            client = form.save(commit=False)
            client.created_by = request.user
            client.team = team
            client.save()
            messages.success(request, "Client created successfully.")
            return redirect('clients:list')
    else:
        form = AddClientForm()
    return render(request, 'client/clients_add.html', {
        'form': form,
        'team': team,
    })

@login_required
def clients_edit(request, pk):
    client = get_object_or_404(Client, created_by=request.user, pk=pk)
    if request.method == 'POST':
        form = AddClientForm(request.POST, instance=client)
        if form.is_valid():
            form.save()
            messages.success(request, "Changes saved successfully")
            return redirect('clients:list')
    else:
        form = AddClientForm(instance=client)
    return render(request, 'client/clients_edit.html', {
        'form': form
    })

@login_required
def clients_delete(request, pk):
    client = get_object_or_404(Client, created_by=request.user, pk=pk)
    client.delete()
    messages.success(request, "Client deleted successfully.")

```

```
return redirect('clients:list')
```

```
from django.urls import path
from . import views

app_name = 'team'

urlpatterns = [
    path('', views.teams_list, name='list'),
    path('<int:pk>/activate/', views.teams_activate, name='activate'),
    path('<int:pk>/edit/', views.edit_team, name='edit'),
    path('<int:pk>/', views.detail, name='detail'),
]
```

```
from django.contrib.auth.models import User
from django.db import models

class Plan(models.Model):
    name = models.CharField(max_length=50)
    price = models.IntegerField()
    description = models.TextField(blank=True, null=True)
    def __str__(self):
        return self.name

class Team(models.Model):
    plan = models.ForeignKey(Plan, related_name='teams', blank=True, null=True,
on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    members = models.ManyToManyField(User, related_name='teams')
    created_by = models.ForeignKey(User, related_name='created_teams',
on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

```
from django import forms
from .models import Team

class TeamForm(forms.ModelForm):
    class Meta:
        model = Team
        fields = ('name',)
```

```
from .models import Team

def active_team(request):
    if request.user.is_authenticated:
        active_team = request.user.userprofile.active_team

        if not active_team:
            active_team = Team.objects.filter(created_by=request.user)[0]
    else:
        active_team = None

    return {'active_team': active_team}
```

```
from django.contrib import messages
```

```
from django.contrib.auth.decorators import login_required
```

```

from django.shortcuts import render, get_object_or_404, redirect
from .forms import TeamForm
from .models import Team
@login_required
def teams_list(request):
    teams = Team.objects.filter(members__in=[request.user])

    return render(request, 'team/teams_list.html', {'teams': teams})

@login_required
def teams_activate(request, pk):
    team = Team.objects.filter(members__in=[request.user]).get(pk=pk)

    userprofile = request.user.userprofile
    userprofile.active_team = team
    userprofile.save()

    return redirect('team:detail', pk=pk)

@login_required
def detail(request, pk):
    team = get_object_or_404(Team, members__in=[request.user], pk=pk)

    return render(request, 'team/detail.html', {'team': team})

@login_required
def edit_team(request, pk):
    team = get_object_or_404(Team, created_by=request.user, pk=pk)

    if request.method == 'POST':
        form = TeamForm(request.POST, instance=team)

        if form.is_valid():
            form.save()

            messages.success(request, 'The changes was saved!')

            return redirect('userprofile:myaccount')
    else:
        form = TeamForm(instance=team)

    return render(request, 'team/edit_team.html', {
        'team': team,
        'form': form,
    })

```

6. Challenges Faced:

- Transitioning to class-based views from function-based views in the leads app to enhance code reusability and structure while ensuring efficient code refactoring.
- Applying TailwindCSS classes to customize and improve the user interface of authentication forms, ensuring responsive and visually appealing designs.

7. Future Enhancements:

- ✓ Integration of payment methods for teams and add files for leads.
- ✓ Integration with smtp gmail services for Email verification of users and enhance user authentication through implementing password resetting functionalities.
- ✓ Enhanced Team features so that a team owner can manage members of their team.

8. Conclusion:

The "Customer Relationship Management using Django" project successfully demonstrates the development of a robust, scalable web application that helps businesses manage customer interactions and data efficiently. Leveraging Django's powerful framework and TailwindCSS for styling, the project implements key functionalities such as lead management, user authentication, and responsive UI design. By utilizing class-based views for cleaner code structure and integrating PDF generation for reporting, this CRM system offers a streamlined solution for enhancing customer relationships and improving business productivity.

9. References:

1. Django Documentation: [<https://docs.djangoproject.com/en/5.1/>]
2. TailwindCSS Documentation: [<https://v2.tailwindcss.com/docs>]
3. Stack Overflow: [<https://stackoverflow.com>]
4. ChatGPT
5. Youtube